

```

//mamal.h

#ifndef MAMMAL_H
#define MAMMAL_H

#include <iostream> // Include for I/O operations
#include <cstdlib> // Include for general purpose functions such as rand()
#include <ctime> // Include for time functions
#include <string> // Include for string operations
#include <vector> // Include for using vector container
#include <fstream> // Include for file operations

using namespace std;

// Define the base class Mammal
class Mammal {
protected:

    string gender_obj; // Attribute to store gender of the mammal

    int age_obj; // Attribute to store age of the mammal

public:

    // Constructor initializes Mammal with gender and age
    explicit Mammal(string gender, int age) : gender_obj(gender), age_obj(age) {}

    // Randomly assigns gender to the mammal as either 'male' or 'female'
    void setGender() {
        int random = rand() % 2; // Generate a random number (0 or 1)
        gender_obj = (random == 0) ? "male" : "female"; // Set gender based on random number
    }
}

```

```

// Sets the age of the mammal
void setAge(int age) {
    age_obj = age; // Directly set age_obj to the given age parameter
}

// Increments the age of the mammal by one year
void incrementAge() {
    age_obj++; // Increase age_obj by 1
}

// Returns the gender of the mammal
string getGender() const {
    return gender_obj; // Return the value of gender_obj
}

// Returns the age of the mammal
int getAge() const {
    return age_obj; // Return the value of age_obj
}
};

#endif // MAMMAL_H

//cat.h

#ifndef CAT_H
#define CAT_H

```

```

#include "mammal.h" // Include the base class header, Mammal, for inheritance

#include <iostream>

#include <cstdlib> // Include for using rand() function

#include <ctime>

#include <string>

#include <vector>

#include <fstream> // Include for file operations


// Cat class inherits from Mammal class
class Cat: public Mammal {
protected:

    string color_obj; // Attribute to store the color of the cat
    string name_obj; // Attribute to store the name of the cat
    int marvelpower_obj; // Attribute to store the Marvel power level of the cat


public:

    // Constructor to initialize Cat object with specified attributes
    explicit Cat(string gender, int age, string color, string name, int marvelpower)
        : Mammal(gender, age), color_obj(color), name_obj(name), marvelpower_obj(marvelpower) {}


    // Sets the color of the cat
    void setColor(const string& color) {
        color_obj = color;
    }


    // Returns the current color of the cat
    string getColor() const {
        return color_obj;
    }
}

```

```
// Randomizes the color of the cat using predefined color options
void randomizeColor() {
    const vector<string> colors = {"white", "brown", "grey", "black", "blonde"}; // Predefined colors
    int random_index = rand() % colors.size(); // Generate a random index
    setColor(colors[random_index]); // Set the cat's color to the randomly selected color
}
```

```
// Sets the name of the cat
void setName(const string& name){
    name_obj = name;
}
```

```
// Returns the current name of the cat
string getName() const {
    return name_obj;
}
```

```
// Randomizes the name of the cat by reading from a file containing names
void randomizeNameFromFile() {
    fstream file;

    file.open("names.txt", ios::in); // Open the names file

    vector<string> names; // Vector to hold the names from the file
    string temp;
    while (file >> temp) { // Read names into temp and push into names vector
        names.push_back(temp);
    }
}
```

```

    if (!names.empty()) {
        int random = rand() % names.size(); // Generate a random index
        setName(names[random]); // Set the cat's name to the randomly selected name
    }

    file.close(); // Close the file after reading names
}

// Randomly sets the Marvel power status using a random number
void setMarvelPowerRandomly() {
    int marvelpower = rand() % 100 + 1; // Generate a random number between 1 and 100
    marvelpower_obj = marvelpower; // Set the marvelpower attribute to the generated number
}

// Determines if the cat has a Marvel power (true if power is <= 2)
bool getMarvelPower() const {
    return marvelpower_obj <= 2; // Returns true if marvelpower_obj is 2 or less
}

};

#endif // CAT_H

//process.h

#ifndef PROCESS_H
#define PROCESS_H

#include "cat.h"

```

```

#include <iostream>

#include <cstdlib>

#include <ctime>

#include <string>

#include <vector>

#include <fstream>

#include <algorithm>

#include <random> // Include for access to random number generation and shuffling utilities


using namespace std;


class Process {
protected:
    vector<Cat> cats; // Stores all Cat instances
    vector<string> names; // Stores names of cats
    vector<string> colors; // Stores colors of cats
    vector<int> ages; // Stores ages of cats
    vector<bool> marvelPowers; // Stores Marvel power status of cats (true if has power)
    vector<string> genders; // Stores genders of cats
    default_random_engine rng; // Random number generator for various random operations

public:
    // Constructor to initialize the Process with a single Cat
    explicit Process(string gender, int age, string color, string name, int marvelpower)
        : cats{Cat(gender, age, color, name, marvelpower)}, rng(static_cast<unsigned>(time(nullptr))) {}

    // Sets up the initial conditions or genesis for the simulation
    void setGenesis() {
        ofstream outFile("output.txt", ios::app); // Open output file in append mode to log events
    }

```

```

    for (int i = 0; i < 5; i++) {

        Cat newCat("unknown", 0, "unknown", "unknown", 0); // Create new cats with unknown
properties

        newCat.randomizeNameFromFile(); // Randomize their names from a file

        newCat.randomizeColor(); // Randomize their color

        newCat.setAge(0); // Set their age to 0

        newCat.setGender(); // Randomly set their gender

        newCat.setMarvelPowerRandomly(); // Randomly decide if they have a Marvel power


        cats.push_back(newCat); // Add new cat to the vector of cats

        storeCatProperties(newCat); // Store cat properties in separate vectors


        outFile << (newCat.getMarvelPower() ? "Marvel Kitty " : "Kitty ") << newCat.getName() << " is
born!" << endl;

    }

    outFile.close(); // Close the output file

}


// Helper function to store properties of cats into separate vectors
void storeCatProperties(const Cat& cat) {

    names.push_back(cat.getName());

    colors.push_back(cat.getColor());

    ages.push_back(cat.getAge());

    marvelPowers.push_back(cat.getMarvelPower());

    genders.push_back(cat.getGender());

}


// Ages all cats by one year and manages their life status and effects
void ageAllCatsByOneYear() {

```

```

ofstream debugLog("debugLog.txt", ios::app); // Open debug log file in append mode
ofstream outFile("output.txt", ios::app); // Open output file in append mode

vector<size_t> toRemove; // Indices of cats that will be removed due to age or conditions
vector<size_t> marvelCats; // Indices of cats with Marvel powers

for (size_t i = 0; i < ages.size(); ++i) {
    ages[i]++; // Increment age of each cat
    if (marvelPowers[i]) {
        marvelCats.push_back(i); // If cat has Marvel power, add to marvelCats list
        debugLog << "Debug: Cat " << names[i] << " (Marvel Power) is now " << ages[i] << " years
old." << endl;
    }
    if (ages[i] >= 10 && (!marvelPowers[i] || ages[i] >= 25)) { // If cat reaches age limit
        debugLog << names[i] << " passes away." << endl;
        outFile << (marvelPowers[i] ? "Marvel Kitty " : "Kitty ") << names[i] << " passed away." <<
endl;
        toRemove.push_back(i);
    } else if (marvelPowers[i]) {
        debugLog << "Debug: Cat " << names[i] << " with Marvel Power survives another year." <<
endl;
    }
}

```

```

removeCats(toRemove); // Remove deceased cats from the lists

```

```

createNewCats(); // Create new cats based on remaining cats

```

```

vector<size_t> potentialTargets; // Indices of cats that can potentially gain Marvel powers

```

```

for (size_t i = 0; i < marvelPowers.size(); ++i) {

```

```

    if (!marvelPowers[i]) {

```

```

        potentialTargets.push_back(i);
    }
}

// Marvel cats infect one non-Marvel cat with Marvel power
for (size_t index : marvelCats) {
    if (!potentialTargets.empty()) {
        size_t targetPosition = rand() % potentialTargets.size(); // Select a random target
        size_t targetIndex = potentialTargets[targetPosition];
        marvelPowers[targetIndex] = true; // Infect the target with Marvel power

        debugLog << "Debug: Cat " << names[index] << " with Marvel Power has infected " <<
names[targetIndex] << "." << endl;

        potentialTargets.erase(potentialTargets.begin() + targetPosition); // Remove target from
potential targets
    }
}

debugLog.close(); // Close the debug log
outFile.close(); // Close the output file
}

// Creates new cats based on breeding eligible male and female cats
void createNewCats() {
    ofstream outFile("output.txt", ios::app);

    vector<int> maleIndices, femaleIndices; // Collect indices for eligible males and females

    // Identify eligible males and females based on age and lack of Marvel power
    for (size_t i = 0; i < genders.size(); ++i) {
        if (ages[i] >= 2 && !marvelPowers[i]) { // Cats must be old enough and not have Marvel Power

```

```

    if (genders[i] == "male") {
        maleIndices.push_back(i);
    } else if (genders[i] == "female") {
        femaleIndices.push_back(i);
    }
}
}
}

```

// Iterate over each eligible female and pair with each eligible male to create new cats

```
for (int femaleIndex : femaleIndices) {
```

```
    string motherColor = colors[femaleIndex]; // Use the mother's color for the kitten
```

```
    for (int j = 0; j < maleIndices.size(); j++) {
```

```
        Cat newCat("unknown", 0, motherColor, "NewCat" + to_string(cats.size()), false);
```

```
        newCat.randomizeNameFromFile(); // Assign a name from file
```

```
        newCat.setGender(); // Randomly set gender
```

```
        newCat.setMarvelPowerRandomly(); // Correctly set Marvel power randomly
```

```
        cats.push_back(newCat);
```

```
        storeCatProperties(newCat);
```

```
    // Output the birth announcement with the correct Marvel status
```

```
    outFile << (newCat.getMarvelPower() ? "Marvel Kitty " : "Kitty ") << newCat.getName() << " is  
born!" << endl;
```

```
    }
```

```
}
```

```
outFile.close(); // Always ensure to close the file
```

```
}
```

```

// Remove specified cats from all vectors by indices
void removeCats(const vector<size_t>& toRemove) {
    if (toRemove.empty()) return;

    for (auto it = toRemove.rbegin(); it != toRemove.rend(); ++it) {
        if (*it < cats.size()) {
            cats.erase(cats.begin() + *it);
            names.erase(names.begin() + *it);
            colors.erase(colors.begin() + *it);
            ages.erase(ages.begin() + *it);
            marvelPowers.erase(marvelPowers.begin() + *it);
            genders.erase(genders.begin() + *it);
        }
    }
}

// Culls half the population of cats randomly
void cullPopulation() {
    ofstream debugLog("populationCullLog.txt", ios::app);
    ofstream outFile("output.txt", ios::app);

    debugLog << "Starting population cull. Initial count: " << cats.size() << endl;

    if (cats.size() < 2) {
        debugLog << "Not enough cats to perform culling. No action taken." << endl;
        outFile << "Not enough cats to perform culling. No action taken." << endl;
        debugLog.close();
        outFile.close();
    }
}

```

```

        return;
    }

    size_t numberToCull = cats.size() / 2; // Calculate half the population
    vector<size_t> indicesToCull(cats.size());
    iota(indicesToCull.begin(), indicesToCull.end(), 0); // Generate a list of indices
    shuffle(indicesToCull.begin(), indicesToCull.end(), rng); // Shuffle the indices

    indicesToCull.resize(numberToCull); // Resize to number to cull

    sort(indicesToCull.begin(), indicesToCull.end(), greater<size_t>()); // Sort in descending order
    for safe removal

    debugLog << "Indices to cull (sorted in descending): ";
    for (size_t idx : indicesToCull) debugLog << idx << " ";
    debugLog << endl;

    for (size_t index : indicesToCull) {
        if (index < cats.size()) {
            debugLog << "Culling Cat - Name: " << names[index] << ", Age: " << ages[index] << ", Color: "
<< colors[index] << ", Marvel Power: " << (marvelPowers[index] ? "True" : "False") << endl;

            outFile << (marvelPowers[index] ? "Marvel Kitty " : "Kitty ") << names[index] << " passed
away." << endl;

            cats.erase(cats.begin() + index);
            names.erase(names.begin() + index);
            colors.erase(colors.begin() + index);
            ages.erase(ages.begin() + index);
            marvelPowers.erase(marvelPowers.begin() + index);
            genders.erase(genders.begin() + index);
        }
    }
}

```

```

    debugLog << "Population after cull: " << cats.size() << endl;
    debugLog.close();
    outFile.close();
}

// Displays all cats' properties formatted for a specific year
void displayAllCats(int year) {
    ofstream outFile("output.txt", ios::app);
    if (!outFile.is_open()) {
        cerr << "Failed to open output.txt for writing." << endl;
        return;
    }
    outFile << "\nAfter aging all cats by " << year << " year(s):" << endl;
    for (size_t i = 0; i < names.size(); ++i) {
        outFile << "Cat: " << names[i]
            << ", Gender: " << genders[i]
            << ", Color: " << colors[i]
            << ", Age: " << ages[i]
            << ", Marvel Power: " << (marvelPowers[i] ? "True" : "False") << endl;
    }
    outFile.close();
}

// Checks if there are any cat names in the vector, used to check for existence of cats
bool hasCatNames() const {
    return !names.empty();
}
};

```

```
#endif // PROCESS_H
```

```
//main.cpp
```

```
#include "Process.h" // Include the header file for the Process class
```

```
#include <cstdlib> // Include the standard library for general utilities
```

```
#include <ctime> // Include the library for time functions
```

```
#include <iostream> // Include the library for input and output stream
```

```
#include <fstream> // Include the library for file input/output operations
```

```
int main(){
```

```
    srand(time(0)); // Seed the random number generator with the current time
```

```
    Process myProcess("uk", 5, "uk", "uk", 0); // Instantiate a Process object with initial parameters
```

```
    myProcess.setGenesis(); // Initialize or set up initial conditions for the process
```

```
    myProcess.displayAllCats(0); // Display the status of all cats at year 0
```

```
    int year = 1; // Start counting years from 1 after genesis
```

```
    while (true) { // Enter an infinite loop to process each year
```

```
        cout << "Do you wish to continue? Press 1. Or kill half the population? Press 2. Or end the  
program? Press any other button: ";
```

```
        int choice = 0;
```

```
        cin >> choice; // Read user's choice from standard input
```

```
        if (choice == 1){
```

```
            myProcess.ageAllCatsByOneYear(); // Age all cats by one year without any population control
```

```
        } else if (choice == 2){
```

```
            myProcess.ageAllCatsByOneYear(); // Age all cats by one year
```

```
        myProcess.cullPopulation(); // Execute a function to reduce the population by half
    } else {
        cout << "Ending program." << endl;
        break; // Exit the loop and end the program if the user chooses any option other than 1 or 2
    }

    myProcess.displayAllCats(year); // Display the status of all cats for the current year

    if (!myProcess.hasCatNames()) { // Check if there are no cats remaining
        ofstream outFile("output.txt", ios::app); // Open or create output file in append mode
        outFile << "All Cats Are Dead!!!" << endl; // Write to file that all cats are dead
        outFile.close(); // Close the file
        break; // Exit the loop since no cats are left
    }

    year++; // Increment year at the end of the loop for the next iteration
}

return 0; // Return 0 to indicate successful completion of the program
}
```